

Fabricio Rodriguez

Baltimore, MD | 240-439-5354 | frodrig3@umbc.edu | [linkedin.com/in/fabricio-rodriguez-816676255](https://www.linkedin.com/in/fabricio-rodriguez-816676255) | github.com/rodriguezzfabricio

EDUCATION

University of Maryland, Baltimore County

B.S. in Computer Science, GPA: 3.1

Baltimore, MD

Aug. 2022 – Dec. 2026

EXPERIENCE

The Walt Disney Company

Burbank, CA

Software Engineering Intern, Studio Technology (AI Platform)

June 2026 – Aug. 2026

- Owned the member-facing model catalog on the studio's internal LLM gateway — browse, detail, and compare views in **TypeScript**, **React 19**, and **Next.js** App Router, resolving through one server-side loader so three views of a model can never disagree
- Separated model identity from routing provider after five non-OpenAI model families inherited OpenAI branding through OpenAI-compatible endpoints; resolved model family before provider inference and used neutral marks where no approved artwork existed
- Closed a data exposure where permission-scoped access fields and admin metadata reached catalog responses served to unprivileged users — stripped at the serialization boundary, enforced at three independent points, with tests asserting the serialized JSON contains none of them
- Designed a model access-request service — **PostgreSQL** schema with audit history, user and admin REST APIs, permission-gated review queue — where approval records the decision and a human still performs the grant, keeping model-spend permissions outside the application's blast radius
- Recovered seven approved merge requests stranded by a closed base branch, replaying ten tickets onto main against a taxonomy upstream had changed twice mid-flight; wrote 12 of the 13 tickets I shipped, and presented the work at the team intern showcase

Financial Industry Regulatory Authority (FINRA)

Rockville, MD

Software Engineer Intern

June 2025 – Aug. 2025

- Defined what “business impact” meant for an **LLM** contract-review agent — prior terms vs. new terms vs. downstream policy effect — and built the component that called the model through an internal AI gateway; team won 1st place at an internal 48-hour hackathon
- Categorized ~500 helpdesk tickets in **Python** and found the top recurring issues were knowledge-availability failures — the answer already existed and users could not find it — which redirected what a company-wide chatbot ingested; shipped the analysis as a Power BI dashboard
- Evaluated that chatbot's recall on newly ingested topics, and built synthetic monitoring for it after finding it exposed no vendor health endpoint: automation drove the real user path and alerted the team channel on failure, running unattended for four weeks and catching a Saturday outage

American University

Washington, DC

Software Engineer Intern

Jan. 2024 – Sep. 2024

- Owned the API contract between a housing-market model built by researchers and a frontend built by another team — **Java**, **Spring Boot**, and PostgreSQL exposing health, inference, and ingest endpoints — and instrumented the usage logging that measured 40–50 daily users

PROJECTS

Job Application Engine | Python, Anthropic API, Next.js 15, React 19, TypeScript, SQLite, GitHub Actions

2026

- Built an LLM tailoring pipeline that structurally cannot invent facts: the model returns schema-validated JSON in which every bullet must resolve to an ID in an approved fact base, then clears five deterministic checks — bullet mapping, number provenance, verb tier, scope guard, noun subset — before a run is marked ok
- Wrote the adversarial suite that keeps it honest: 18 tests each plant one fabrication — invented number, denominator swap, verb inflation, scope drift, subject swap — and fail the build if the matching check does not fire. 76 tests, CI over the Python core and the TypeScript dashboard
- Built the ingestion pipeline over three job-board APIs (Greenhouse, Ashby, Lever): 607 postings across 49 boards, recording the exact phrase behind every flagged requirement and de-duping on a company/title/requisition hash

Depth Engine | AI agent research protocol — prompt architecture, evaluation design

2026

- Designed a 12-stage, 5-law protocol taking an AI agent from a vague goal to a build-ready scaffold, a decision, or a reasoned refusal: hard gates it cannot self-certify past, four-verdict convergence routing, append-only memory, and a named countermeasure for each of 12 known LLM failure modes (context rot, sycophancy, reward-hacking the verifier)
- Ran it on a real question and audited its own output: an adversarial pass caught a hallucinated arXiv citation the agent had produced, plus four more sourcing defects found by re-fetching every source first-party instead of trusting a summarizing fetch. Turned those failures into three standing verification rules and 22 fault-injection tests, one per failure mode

TECHNICAL SKILLS

Languages: TypeScript, JavaScript, Python, Java, Go, SQL (Postgres), HTML/CSS

Frameworks: React, Next.js (App Router, server components), React Native (Expo), Spring Boot, Gin, Tailwind, Zod

AI/LLM: Anthropic API (CLI and SDK), prompt architecture, schema-constrained model output (Zod, JSON Schema), adversarial evaluation and fault-injection testing of model output, LLM gateway/model routing

Backend & Data: REST API design, OpenAPI contract checks, PostgreSQL, SQL migrations, Supabase, Stripe

Testing & Tools: pytest, Vitest, Node test runner, Playwright, Selenium, Git, GitHub Actions, GitLab CI, Jira, Figma, Power BI